DIFX GPU CORRELATOR · ADELLER2023B_DIFX

DiFX GPU Benchmarking Runbook

How to measure the GPU correlator — on a desktop with one card, and on the OzSTAR A100 nodes. Every command here runs from a clean checkout of the `adam-performance-gains` branch.

Branch adam-performance-gains **Verified at** 5ba5d045c **Updated** 2026-10-12

The two harnesses, and which question each answers

They are not interchangeable, and using the wrong one is how this project has produced its wrong conclusions.

WHAT TO REACH FOR

HARNESS	ANSWERS	DOES NOT ANSWER
`tests/Synthetic`	**Is the GPU path still correct?** Correlates each scenario on CPU and GPU and diffs the visibilities.	Anything about speed — the scenarios are tiny.
`tests/FakeData`	**Is it faster?** A 10-station, 16 × 32 MHz, 2 Gbps/station job using `source = fake`, so no baseband data is needed.	Correctness. The fake streams are pathological by design — never gate on them.

THE RULE THAT COSTS THE MOST WHEN IGNORED

If the effect you are chasing is a few percent of one kernel, measure the kernel — not a correlation. A wall-clock A/B on a shared cluster node once returned the wrong *sign* on a 5% kernel win, because per-run launch and teardown cost 2–3.5 s of a 24 s leg. The kernel-level sweep settled the same question in two minutes and is immune to co-tenants.

One-time setup

1 Get the branch

```
git clone git@github.com:ADACS-Australia/ADeller2023B_difx.git
cd ADeller2023B_difx
git checkout adam-performance-gains
```

2 Point `setup.bash` at your machine

Copy the repo-root `setup.bash` and edit `DIFXROOT`, `IPPROOT`, `MPICXX`, `CUDAROOT` and `NVCCFLAGS` (`-arch=` for *your* card: `sm_75` Turing, `sm_80` A100, `sm_89` Ada). Source it in every shell that runs DiFX.

```
source setup.bash
```

3 Build with CUDA

```
./install-difx --withcuda --makeflags='-j 8'
```

For later rebuilds of just the correlator, `./rebuild-mpifxcorr.sh 8` does clean → make → **make install** in one step. The `make install` is not optional: a stale installed binary masquerading as a new build has cost this project entire sessions.

4 Verify you are running what you think you are

```
ls -l $(which mpifxcorr)                                  # newer than your
strings $(which mpifxcorr) | grep DIFX_GPU_FRINGE_TILE # gate present =
```

5 Generate the synthetic test data

Only `tests/Synthetic` needs real files; FakeData fabricates its own. This writes ~2 GB of VDIF and takes a few minutes.

```
cd tests/Synthetic && ./createData.sh
```

6 Pre-generate the 2-station profiling job

`run-bench.sh` builds its own jobs, but the two profiling harnesses expect `benchprof2_1.input` to exist already. Do this once:

```
cd tests/FakeData
vex2difx benchprof2.v2d && difxcalc benchprof2_1.calc
```

Running locally

All four of these run on a single desktop GPU. Start with correctness; a performance number from an incorrect build is worth nothing.

Correctness — CPU vs GPU, every scenario

```
cd tests/Synthetic
./run-local.sh                    # all scenarios
./run-local.sh usb dualpol        # just these two
```

Each scenario is correlated three times — CPU, GPU with `DIFX_GPU_PIPELINE=0`, GPU with `=1` — and the raw visibilities are diffed. **Both GPU columns must read PASS.** The LSB scenarios report SKIP: the GPU path declines lower sideband by design.

Expect ~10 minutes for the full set. Diff logs land under `local-runs/<scenario>/diff/`.

A TRAP IN THE TOOLING

`diffDiFX.py` **always exits 0.** It signals problems only by printing `THRESHOLD EXCEEDED!` or header-disagreement lines, which the harness parses for you. Never gate a script on its exit code.

Throughput — the ledger metric

```
cd tests/FakeData
./run-bench.sh 3                    # 3 repeats, best-of-3 reported
USEGPU=0 ./run-bench.sh 3           # CPU path for comparison
EXTRA_ENV="DIFX_GPU_FRINGE_TILE=0" ./run-bench.sh 3
```

The metric is **T5 − T1**: wall time of a 5 s correlation minus a 1 s one, which cancels the large constant start-up and shutdown costs and leaves the steady-state rate. It prints a ready-made row for `BENCHMARKS.md`.

It defaults to rewriting the job to **single-thread VDIF**, bypassing the DataStream interlaced-VDIF corner-turn. That is deliberate: the corner-turn is CPU-bound at ~1.7 Gbps/core and starves the GPU, so leaving it in measures the demultiplexer rather than the correlator. Set `SINGLE_THREAD_VDIF=0` only when the demux is what you are studying.

Per-kernel A/B across many data shapes

```
cd tests/FakeData
./fringetile-sweep.sh 2500 30                  # real sampling
SWEEP_COMPLEX=1 ./fringetile-sweep.sh 2500 30  # complex twin
SWEEP_PCAL=1    ./fringetile-sweep.sh 2500 20  # + the phase-cal path
```

This drives the *shipped* fused decode+fringe launcher over 15 (bands, channels) shapes from 1×64 to 128×128, on both the tiled and untiled paths, and prints per-shape speedup plus an output hash. Because both paths do identical arithmetic in identical order, **equal hashes are an exact correctness check** — covering odd band counts and partial tiles no DiFX scenario exercises.

Pass `2500 30`, not the defaults. At the default 10 windows the launch is latency-dominated and every shape reads ~1.0×; below ~30 repeats the largest shape swings 18% on warm-up alone.

Profiles — what a kernel is bound by

```
cd tests/FakeData
./nsys-frac.sh <tag>                          # timeline + kernel mix
./ncu-frac.sh  <tag> metrics                  # cheap fixed metric list
./ncu-frac.sh  <tag> full 'gpu_fused_fringe'  # + warp-stall reasons
```

`ncu` needs unrestricted GPU performance counters, or it fails with `ERR_NVGPUCTRPERM`. Check with `grep RmProfilingAdminOnly /proc/driver/nvidia/params` — you want `0`. If it reads `1`, add `options nvidia NVreg_RestrictProfilingToAdminUsers=0` to `/etc/modprobe.d/` and reboot.

Keep `full` to one or two launches: it replays each roughly twenty times.

Running on OzSTAR

Every script below has an editable config block in its first ~20 lines — `SETUP`, `SRCDIR`, `WORKDIR`. Point `SETUP` at your own DiFX setup script and `WORKDIR` at your own scratch under `/fred` before the first submission.

SUBMIT FROM TESTS/FAKEDATA

SCRIPT	GIVES	NEEDS THE NODE?
`benchprof-profile-nsys-5s.sbatch`	A 5 s Nsight Systems capture — kernel mix, µs/call, GPU busy/idle, NVTX phases.	No — per-kernel figures shrug off co-tenants.
`benchprof-profile-nonsys-20s.sbatch`	A 20 s bare run. Its whole output is an absolute wall time.	**Yes.**
`benchprof-fringetile-ab.sbatch`	An interleaved A/B of two gate states in one allocation.	Preferably — but read the caution below.
`benchprof-cpu-20s.sbatch`	The CPU-only baseline. Sweep threads with `--export=ALL,NTHREAD=40`.	Yes, for a quotable number.

Reserving the node

```
sbatch --cpus-per-task=5 benchprof-profile-nonsys-20s.sbatch   # ledger-quality
sbatch                   benchprof-profile-nonsys-20s.sbatch   # quick look on
```

OzSTAR rejects `--exclusive`, so isolation comes from reserving most of the node: `--cpus-per-task=5` × 12 tasks = 60 of gina's 64 cores. It does **not** start more MPI ranks — that is `--ntasks`; the extra cores are simply held. A co-tenant job stalls the input copies and costs ~10% of wall with no other symptom, so every script prints a `node ownership: N/M cores` verdict. Read it before quoting any wall time.

KEEP --NODES=1

Without it, SLURM satisfies a wide `--cpus-per-task` by splitting the twelve ranks across two nodes. `--gres` is a *per-node* request, so that reserves a GPU on each and uses one, and DataStream→Core traffic leaves intra-node shared memory for the fabric. All the scripts pin it and warn if it is overridden.

Two captures at once

```
sbatch --cpus-per-task=5 --export=ALL,PROFILE_TAG=ac0,DIFX_GPU_XMAC_AUTOCORR=0
sbatch --cpus-per-task=5 --export=ALL,PROFILE_TAG=ac1,DIFX_GPU_XMAC_AUTOCORR=1
```

`PROFILE_TAG` keeps the two legs in separate workdirs and report names — without it they share a scratch directory that each wipes on startup. Each run echoes its gate settings into the mpilog, so a report can never be mistaken for the other leg.

Getting the numbers out

The login node's `nsys` is older than the captures and cannot export them. Copy the `.nsys-rep` to a machine with a current toolkit and run the repo's own analyser, which prints every summary in one pass:

```
tests/FakeData/nsys-analyze.py <capture>.nsys-rep
```

The kernel-level sweep also runs on OzSTAR, needing only one GPU and a couple of cores — usually a near-instant allocation:

```
srun --gres=gpu:1 --time=15 --mem=8000 --account=oz168 ./fringetile-sweep.sh 2
```

Runtime gates

All read from the environment at startup, so A/B pairs need no rebuild. Pass them through `EXTRA_ENV=` to `run-bench.sh`, or `--export=ALL,VAR=value` to `sbatch`.

MPIFXCORR GPU ENVIRONMENT VARIABLES

VARIABLE	DEFAULT	EFFECT
`DIFX_GPU_PIPELINE`	1	Intra-subint host-tail overlap. `0` is the synchronous path. Both must pass correctness.
`DIFX_GPU_FRINGE_TILE`	on	Shared-memory transpose in the fused decode+fringe kernel. `1.2–2.2×` on both cards tested; `0` restores the old kernel.
`DIFX_GPU_XMAC_AUTOCORR`	off	Autocorrelations computed in the XMAC as ordinary baselines. `7% slower on a 2070` — the A100 verdict is still open.
`DIFX_GPU_WEIGHTS_HOST`	off	Falls back to host-side weight accumulation. A correctness cross-check, not a performance option.
`DIFX_GPU_PIN_INPUT`	on	Page-locked Core receive buffers, so input DMAs skip a host staging copy.

Harness-side knobs: `SINGLE_THREAD_VDIF=0` keeps the interlaced demux; `USEGPU=0` benchmarks the CPU path; `SWEEP_COMPLEX` / `SWEEP_PCAL` select the sweep's sampling mode and phase-cal leg; `SWEEP_SETUP`, `SWEEP_NVCC`, `SWEEP_OUTDIR` and `SWEEP_ARCH` override the sweep's machine detection.

Three findings that shape how you measure

Each of these was learned by reaching a wrong conclusion first.

The two cards disagree, and the answer does not transfer

The kernel mix on an RTX 2070 is nothing like an A100's — `gpu_fused_fringe` is 46.5% of kernel time on one and ~21% on the other. Atomics in the fractional-rotation kernel measured 24% of it on the A100 and **0.0%** on the 2070. A "what is this bound by" answer is a property of the card, so measure on the card you care about, and say which card any number came from.

Benchmark single-threaded, then remember you did

The interlaced-VDIF corner-turn runs at ~1.7 Gbps per core, so a VGOS-rate 4 × 16 Gbps observation would need ~38 cores doing nothing but demultiplexing — more silicon than the correlation. Bypassing it is right for measuring the correlator and *misleading* for specifying hardware.

Stale baselines produce confident nonsense

The autocorrelation work was planned from probe numbers taken before an earlier change had already removed what made that work expensive. The plan predicted a 12–15% win; the measurement showed a 7% loss. Before sizing anything from a recorded number, check what landed after it.

Open measurements

Two cluster runs would each settle a question that is currently blocked. Neither had reported as of 2026-10-12.

RUN	SETTLES	STATUS
Autocorrelation A/B on the A100 — the two `PROFILE_TAG=ac0/ac1` captures above	Whether `DIFX_GPU_XMAC_AUTOCORR` is worth keeping. Compare `gpu_resultsrotatorMultiply` µs/call (what autocorrelations cost today) against `gpu_fuse_xmac_and_average` µs/call (what moving them costs), plus NVTX `host_finalize`.	open
20 s wall run at the current commit	Refreshes the A100 figure behind the cost/benefit write-up, which still predates both the conjugate-array removal and the tiling, and so understates the GPU.	open

Where things are written down

FILE	HOLDS
`docs/gpu-plan.md`	The living plan — where the work stands, what is queued, what was rejected. Canonical, ahead of anything remembered.
`docs/gpu-changes.md`	Prose record, one section per substantive change (settled sections in `gpu-changes-archive.md`).
`tests/FakeData/BENCHMARKS.md`	The numbers ledger, plus the protocol for producing a quotable one.
`docs/gpu-profiling.md`	How to profile, reference captures, and the measurement pitfalls.
`tests/Synthetic/README`, `tests/FakeData/README.md`	Per-harness detail, including every scenario and what it covers.

Every landed change gets a `run-bench.sh` row in `BENCHMARKS.md` and a section in `gpu-changes.md`, written at the time of the change. A measurement that is not written down did not happen.